Frozen LLM Embeddings Need Not Beat TF–IDF for Cuisine Classification from Ingredient Lists

COMP 345 Midterm Report (Checkpoint 2)

Ruoyu Gu
COMP 345 – Fall 2026

30 October 2026

Code and demo notebook: https://github.com/Karmin77/COMP345ProjectCuisine

Abstract

This report documents the system shown in the midterm demo: cuisine classification from ingredient lists with two finished representations, a task-specific TF-IDF baseline and a frozen 768-dimensional `gemini-embedding-001` vector. Both share the same normalized ingredient text, the same group-aware splits, and the same PyTorch linear softmax probe, so the representation is the controlled difference. Over three outer folds at full training, TF-IDF reaches test macro-F1 0.7048 and the frozen embedding 0.6462; the paired difference is -0.0586 with a 95% group-bootstrap interval of $[-0.0652, -0.0523]$. TF-IDF leads at all eight training proportions, and the embedding loses most on small, overlapping cuisines. Yet the two models are not redundant: the embedding alone is correct on 2,025 recipes that TF-IDF misclassifies. The proposal's two fusion methods, probability fusion and feature fusion, test exactly this complementarity; they remain part of the plan and will be reported in the final report.

1 Problem

The input is a recipe's ingredient list and the output is one of 20 cuisine labels. Ingredient lists are short, unordered, and highly lexical: a single ingredient such as *garam masala* or *fish sauce* can be decisive, while staples such as salt, garlic, and olive oil appear everywhere. The project uses this task to ask a representation question rather than to maximize accuracy: *does a frozen LLM embedding of an ingredient list classify cuisines better than classical TF-IDF features when both are fed to the same linear probe?* The research question and hypothesis are those of the project proposal (Version 2). The intended use is scientific analysis of representations, not a production recipe tagger.

2 Hypothesis

A frozen embedding probe will *not* exceed the TF-IDF probe's cuisine-classification macro-F1 by 0.02 or more at the 100% training setting when both use the same linear model class. We evaluate this hypothesis using the paired difference across the three outer test folds and its uncertainty interval, rather than a point estimate alone: an interval entirely below 0.02 supports the hypothesis, an interval crossing 0.02 is inconclusive, and an interval entirely at or above 0.02 is evidence against it.

As in the proposal, learning curves assess whether embeddings help more in low-data settings, and we additionally test whether combining the two models' prediction probabilities (probability fusion) or their features (feature fusion) improves classification.

3 Related approaches

Lexical text classification. Bag-of-words and TF-IDF features with a linear classifier are a long-standing, strong baseline for text classification [1]. They keep exact, task-specific tokens, which suits ingredient lists. We use this family as the baseline.

Frozen representations and linear probes. A linear probe trains only a linear classifier on top of a frozen representation, so its score measures how linearly accessible the task is in that representation [2]. We follow it with a hosted embedding model, `gemini-embedding-001` [5], used through its API as a fixed feature extractor.

Combining representations. Sparse lexical and dense learned features can be combined either early (concatenating features) or late (combining each model's predicted probabilities, e.g. by stacking [3]). These two options are the planned feature-fusion and probability-fusion arms.

What is different here. Instead of maximizing leaderboard accuracy, we hold the classifier, loss, optimizer, splits, and input text fixed and change only the representation, then measure sample efficiency, per-class behaviour, and where the two representations disagree.

4 Datasets

We use the labelled training portion of Kaggle's What's Cooking dataset [4]. The unlabeled Kaggle test file has no cuisine labels and is not used. A single record looks like this:

```
id: 10259   cuisine: greek
ingredients: romaine lettuce, black olives, grape tomatoes, garlic, pepper,
purple onion, seasoning, garbanzo beans, feta cheese crumbles
```

Each ingredient is lowercased, trimmed, and whitespace-normalized; the strings are sorted and joined with newlines. This single serialization is both the TF-IDF document and the Gemini input. Identical normalized lists form a group and never cross a split boundary, which prevents duplicate recipes from leaking between training and test. Fourteen groups (30 recipes) carry conflicting labels; they are retained and reported.

Table 1: Dataset and frozen evaluation setup.

Quantity	Value
Labelled recipes	39,774
Normalized ingredient groups	39,242
Ambiguous groups / recipes	14 / 30
Cuisine classes	20
Outer evaluation	3 stratified, group-aware folds
Validation fraction	15% of each development pool
Training proportions	2, 5, 10, 20, 40, 60, 80, 100%
Random seed	345

Table 2: Recipes per split in each outer fold. Every labelled recipe is a test recipe in exactly one fold.

Fold	Training pool	Validation	Test
0	22,703	3,801	13,270
1	22,731	3,798	13,245
2	22,714	3,801	13,259

5 Models / Workflow

Figure 1 shows the input-to-output path and Figure 2 the model architecture. The blue boxes are finished and shown in the demo; the green box is the next stage and reuses the same splits.

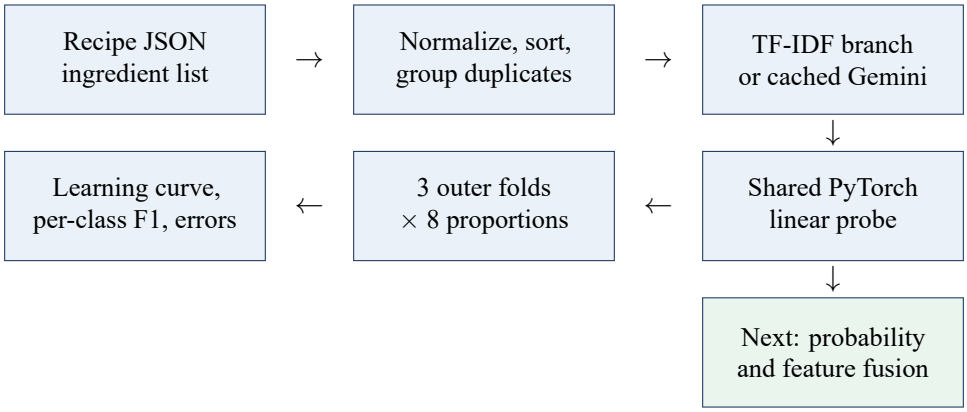


Figure 1: Workflow of the current system. The Gemini API is called only once, to build the embedding cache; every later stage runs locally from saved files.

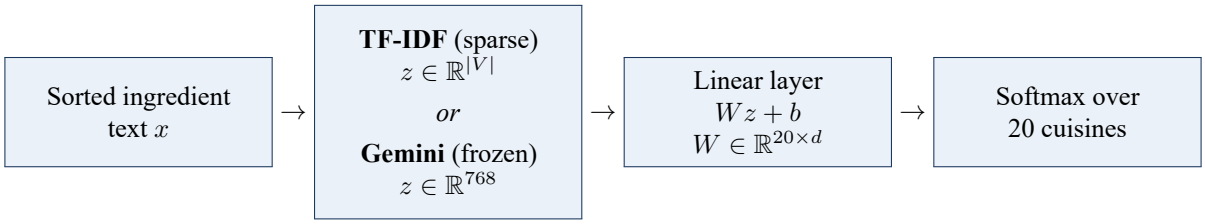


Figure 2: Model architecture. Each run uses one representation z (dimension d); only W and b are trained. $|V|$ is the TF-IDF vocabulary fit on the current training subset.

5.1 Baseline: TF-IDF

The vocabulary and inverse-document-frequency weights are fit on each training subset only. Validation chooses among word unigrams, word unigrams plus bigrams, and whole-ingredient tokens. Across the 24 fold-by-proportion fits, validation chose unigrams plus bigrams 14 times and unigrams 10 times; whole-ingredient tokens were never selected.

5.2 Foundation-model integration: frozen Gemini embedding

Each serialized recipe is sent once to Google's `gemini-embedding-001` with task type `CLASSIFICATION` and 768 output dimensions [5]. The model is not fine-tuned; it is a frozen feature extractor. Generation is a separate, resumable command that writes a local cache. The cache used here has shape $39{,}774 \times 768$ (`float32`), contains only finite values, and is checked against the recipe-id order and a metadata fingerprint before any evaluation reads it. No labels or classifier weights are sent to the API.

5.3 Shared linear probe

For a representation $z(x) \in \mathbb{R}^d$, the probe is

$$f_\theta(z) = \text{softmax}(Wz + b), \qquad W \in \mathbb{R}^{20\times d},\ b \in \mathbb{R}^{20},$$

trained with multiclass cross-entropy and Adam in PyTorch [6]. Learning rate and L2 weight decay are selected on validation macro-F1; training uses batch size 512 and early stopping with patience 8. At full training the selected settings were the same for both representations in every fold (learning rate 10^{-2}, weight decay 10^{-5}). Because the classifier family, loss, optimizer, and selection rule are shared, differences in test scores reflect the input representation.

6 Experiments to test the hypothesis

The proposal's design has four methods (TF-IDF, frozen embedding, probability fusion, feature fusion), each evaluated on the same three outer folds and eight nested training proportions, for 96 evaluation settings. This midterm checkpoint reports the two standalone methods, which give 48 fold-level test scores; the two fusion methods use the same folds and proportions and will be reported in the final report. Test folds are never used for fitting or selection; the validation slice is used only for model and hyperparameter choice.

The primary metric is test macro-F1, which weights the 20 cuisines equally so that small classes are not hidden by Italian and Mexican. Accuracy is reported as a secondary metric. The primary test is the paired difference in macro-F1 between embedding and TF-IDF at 100% training. A group bootstrap with 2,000 replicates resamples whole normalized ingredient groups inside each fold, which preserves the pairing between the two methods' predictions on the same recipes.

7 Results

7.1 Full-training comparison

Table 3: Test results at 100% training, three outer folds.

Method	Mean macro-F1	Fold SD	Fold macro-F1 scores	Mean accuracy
TF-IDF	**0.7048**	0.0053	0.6987, 0.7080, 0.7076	**0.7820**
Frozen Gemini embedding	0.6462	0.0135	0.6397, 0.6372, 0.6617	0.7631

The paired difference, embedding minus TF-IDF, is

$$\Delta = -0.0586, \qquad 95\%\text{ group-bootstrap interval } [-0.0652,\ -0.0523].$$

The whole interval lies below zero and far below the $+0.02$ threshold, so the hypothesis is supported: the frozen embedding does not replace the lexical baseline on this task. The accuracy gap (1.9 points) is smaller than the macro-F1 gap (5.9 points), which already suggests that the embedding's losses are concentrated in smaller classes.

7.2 Learning curve

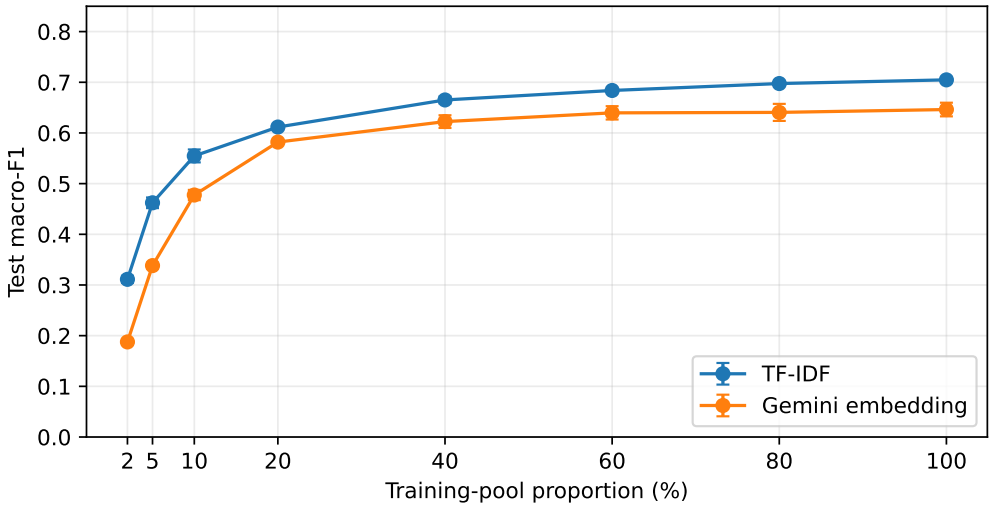


Figure 3: Test macro-F1 against training-pool proportion. Points are means over three outer folds; bars are fold standard deviations.

Table 4: Learning-curve means (test macro-F1).

Training pool	TF-IDF	Frozen embedding	Gap
2%	0.311	0.188	−0.123
5%	0.462	0.338	−0.124
10%	0.555	0.478	−0.077
20%	0.612	0.582	−0.030
40%	0.665	0.622	−0.043
60%	0.684	0.640	−0.044
80%	0.697	0.641	−0.056
100%	0.705	0.646	−0.059

TF-IDF is ahead at every proportion. The answer to the low-data question is therefore negative in this configuration: the pretrained representation does not make the probe more sample-efficient. The gap is widest at 2–5% training, where the smallest cuisines have only a handful to a few dozen training recipes and a 768-dimensional dense probe has too little data to separate them. The embedding curve also flattens after 60%, while TF-IDF keeps improving slowly.

7.3 Per-class F1

Table 5: Mean per-class test F1 at 100% training. Gap is embedding minus TF-IDF; gaps below -0.10 are in bold.

Cuisine	TF-IDF	Gemini	Gap	Cuisine	TF-IDF	Gemini	Gap
Brazilian	0.612	0.393	**−0.219**	Jamaican	0.768	0.655	**−0.113**
British	0.460	0.346	**−0.114**	Japanese	0.744	0.735	−0.009
Cajun Creole	0.725	0.721	−0.004	Korean	0.790	0.719	−0.071
Chinese	0.817	0.808	−0.009	Mexican	0.910	0.909	−0.001
Filipino	0.653	0.585	−0.069	Moroccan	0.795	0.759	−0.036
French	0.615	0.586	−0.029	Russian	0.489	0.382	**−0.107**
Greek	0.750	0.713	−0.037	Southern US	0.745	0.720	−0.025
Indian	0.883	0.878	−0.005	Spanish	0.536	0.456	−0.080
Irish	0.548	0.427	**−0.121**	Thai	0.779	0.750	−0.029
Italian	0.843	0.834	−0.010	Vietnamese	0.632	0.548	−0.084

Large, distinctive cuisines (Mexican, Indian, Italian, Chinese) are nearly tied. The embedding loses most on Brazilian, Irish, British, Jamaican, and Russian: small classes whose identity often rests on one or two rare ingredients that a dense 768-dimensional summary can blur, and which give the probe few examples to learn from.

8 Failure analysis

8.1 Most frequent confusions

Each labelled recipe is a test recipe in exactly one outer fold, so the counts below cover all 39,774 recipes using the saved 100% predictions.

Table 6: Five largest off-diagonal confusions (true → predicted) at 100% training.

TF-IDF		Frozen Gemini embedding	
French → Italian	523	French → Italian	657
Italian → French	339	Greek → Italian	287
Cajun Creole → Southern US	260	Spanish → Italian	279
Southern US → Italian	205	Cajun Creole → Southern US	276
Greek → Italian	199	Italian → French	273

Both models confuse neighbouring cuisines that share ingredients and techniques. The embedding is more strongly pulled toward Italian, the largest Mediterranean class: French, Greek, and Spanish recipes all flow into it more often than under TF-IDF.

8.2 Where the two models disagree

Table 7: Agreement between the two models on all 39,774 test predictions at 100% training.

Outcome	Recipes	Share
Both correct	28,326	71.2%
TF-IDF correct only	2,779	7.0%
Gemini embedding correct only	2,025	5.1%
Both wrong	6,644	16.7%

The embedding is weaker overall, but it is not a subset of TF-IDF: it alone gets 2,025 recipes right. If an oracle could always pick the correct model, accuracy would rise from 78.2% to 83.3%. That ceiling is not achievable, but it shows that there is complementary signal for fusion to look for. Three cases from the demo illustrate the categories:

- **TF-IDF only** (id 22884, British; embedding predicts Indian): *white vinegar, pearl onions, cucumber, ground ginger, flour, ground turmeric, cauliflower, sugar, mustard powder*. This is a piccalilli-style pickle. Turmeric and ginger pull the dense vector toward Indian, while TF-IDF keys on the vinegar/mustard/pearl-onion combination seen in British training recipes.
- **Embedding only** (id 2145, Vietnamese; TF-IDF predicts Thai): *rice vinegar, lime juice, asian fish sauce, hot chili, sugar, garlic cloves*. This is a dipping sauce (nuoc cham). Every token is common in Thai recipes, so lexical weights favour the larger class; the embedding of the whole list lands closer to Vietnamese.
- **Both wrong** (id 8, French; both predict Italian): *whole milk, sea salt, vanilla sugar, lemon, hazelnuts, baking powder, flour, eggs, extra-virgin olive oil*. A hazelnut and olive-oil cake has no ingredient that marks it as French. No representation of this list can recover a label that the list does not contain.

The 6,644 shared errors put a practical ceiling on this task: much of the remaining error comes from overlapping cuisines and from the label itself, not from the choice of representation.

9 Discussion and limitations

Ingredient lists behave close to a bag-of-words problem, so exact lexical evidence is strong, and TF-IDF with bigrams captures informative pairs directly. Compressing a whole list into 768 dense coordinates keeps general culinary similarity but can blur rare, decisive ingredients, which is exactly where the embedding loses (small classes, Mediterranean neighbours). This is a result about one embedding model, one serialization, one probe family, and one dataset, not a claim that foundation-model embeddings are generally worse.

Limitations of this checkpoint: one embedding model and one fixed serialization; one split seed; a linear probe only. Outer-fold training pools overlap, so fold standard deviations are descriptive rather than estimates over independent datasets, and the bootstrap is conditional on the fitted predictions. The 30 ambiguous recipes and the cuisine overlap mean that perfect accuracy is not attainable. Fusion results are not part of this checkpoint, so this report makes no claim about complementarity beyond the disagreement counts in Table 7.

10 Status and plan for the final stretch

Done and shown in the midterm demo. Shared normalization and duplicate grouping; frozen group-aware splits; the TF-IDF baseline; the validated 39,774-recipe Gemini cache; the shared PyTorch probe; the eight-point

learning curve; the paired bootstrap comparison; per-class F1, confusions, and agreement analysis. The demo notebook (`demo/COMP345_midterm_demo.ipynb`) reads only saved files: it makes no Gemini call and trains nothing. Source code, the split manifest, the midterm results tables, and this report are in the GitHub repository (`https://github.com/Karmin77/COMP345ProjectCuisine`). The Kaggle data and the large embedding and prediction archives are not committed; the README explains how to download or regenerate them.

Next, toward the 2 December deliverables.

- **Probability fusion:** combine the two probes' class probabilities with weights learned from group-aware out-of-fold predictions, so no test information enters the combiner.
- **Feature fusion:** concatenate the TF-IDF and embedding features and train the same linear probe, then compare it with both standalone arms on the same folds and proportions.
- **Fusion failure analysis:** count rescue cases (fusion fixes a standalone error) and regressions (fusion breaks a standalone success), and check whether fusion helps the small classes in Table 5.
- **Codebase and presentation:** keep the fresh-clone run in the README working, and prepare the 10-page report, poster, and live demo around the complete experiment.

References

[1] T. Joachims, "Text Categorization with Support Vector Machines: Learning with Many Relevant Features," in *ECML*, 1998.

[2] G. Alain and Y. Bengio, "Understanding Intermediate Layers Using Linear Classifier Probes," arXiv:1610.01644, 2016.

[3] D. H. Wolpert, "Stacked Generalization," *Neural Networks*, 5(2), 241–259, 1992.

[4] Kaggle, "What's Cooking?" Dataset, `https://www.kaggle.com/c/whats-cooking`.

[5] Google, "Gemini API embeddings," `https://ai.google.dev/gemini-api/docs/embeddings`.

[6] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *NeurIPS*, 2019.